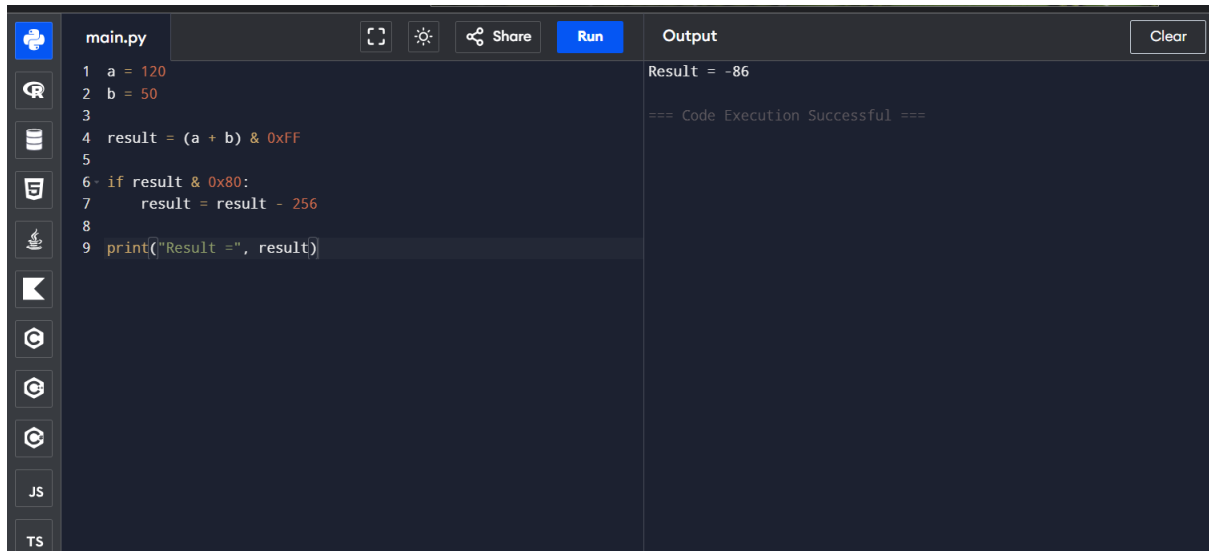


1. Debugging Signed 8-bit Addition:

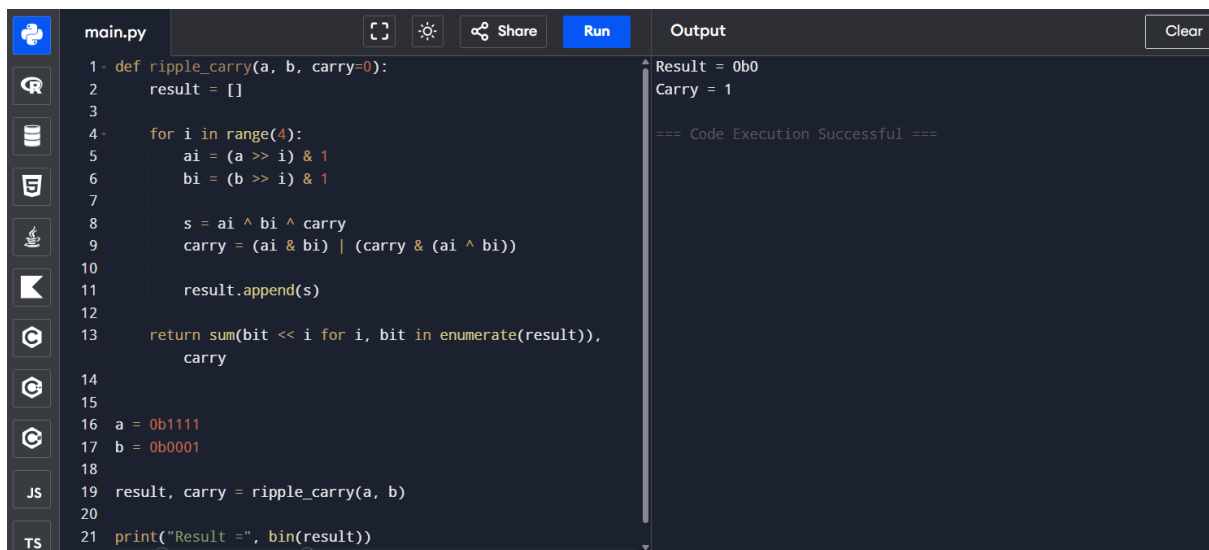


The screenshot shows a code editor with a file named `main.py`. The code defines two variables, `a = 120` and `b = 50`, and calculates `result = (a + b) & 0xFF`. It then checks if the result is negative using `if result & 0x80:` and adjusts it by subtracting 256. Finally, it prints the result. The output window shows the result as `-86` and a success message.

```
1 a = 120
2 b = 50
3
4 result = (a + b) & 0xFF
5
6 if result & 0x80:
7     result = result - 256
8
9 print("Result =", result)
```

Output: Result = -86
=== Code Execution Successful ===

2. Ripple Carry Adder → Carry Look-Ahead:

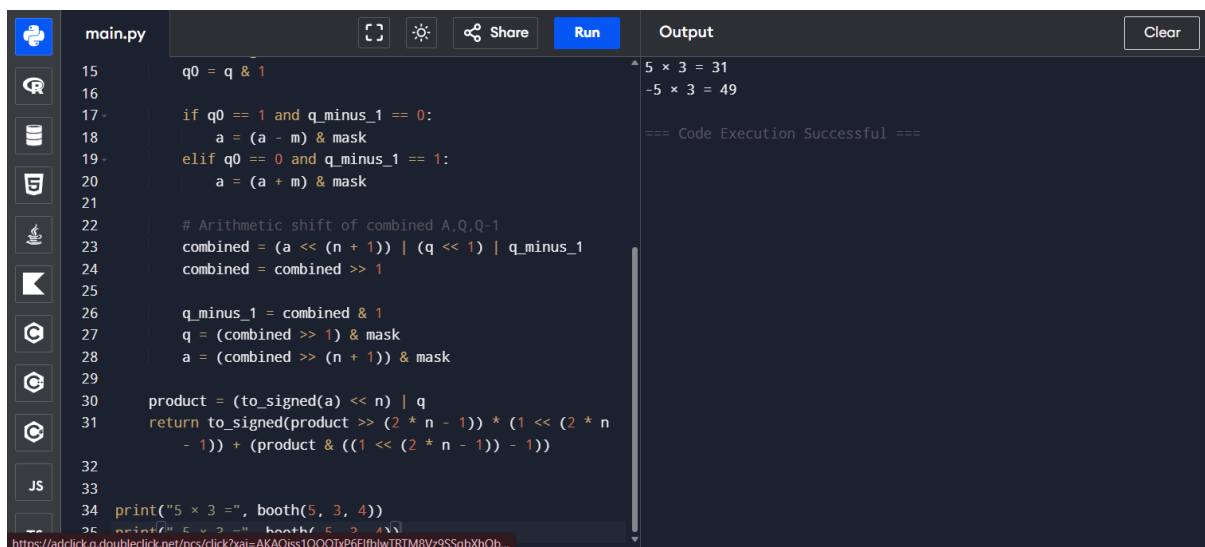


The screenshot shows a code editor with a file named `main.py`. The code defines a function `ripple_carry(a, b, carry=0)` that calculates the sum of two 4-bit numbers `a` and `b` using a carry look-ahead logic. It then calls the function with `a = 0b1111` and `b = 0b0001` and prints the result in binary. The output window shows the result as `0b0` and the carry as `1`, along with a success message.

```
1 def ripple_carry(a, b, carry=0):
2     result = []
3
4     for i in range(4):
5         ai = (a >> i) & 1
6         bi = (b >> i) & 1
7
8         s = ai ^ bi ^ carry
9         carry = (ai & bi) | (carry & (ai ^ bi))
10
11        result.append(s)
12
13    return sum(bit << i for i, bit in enumerate(result)),
14           carry
15
16 a = 0b1111
17 b = 0b0001
18
19 result, carry = ripple_carry(a, b)
20
21 print("Result =", bin(result))
```

Output: Result = 0b0
Carry = 1
=== Code Execution Successful ===

3.Booth's Multiplication:

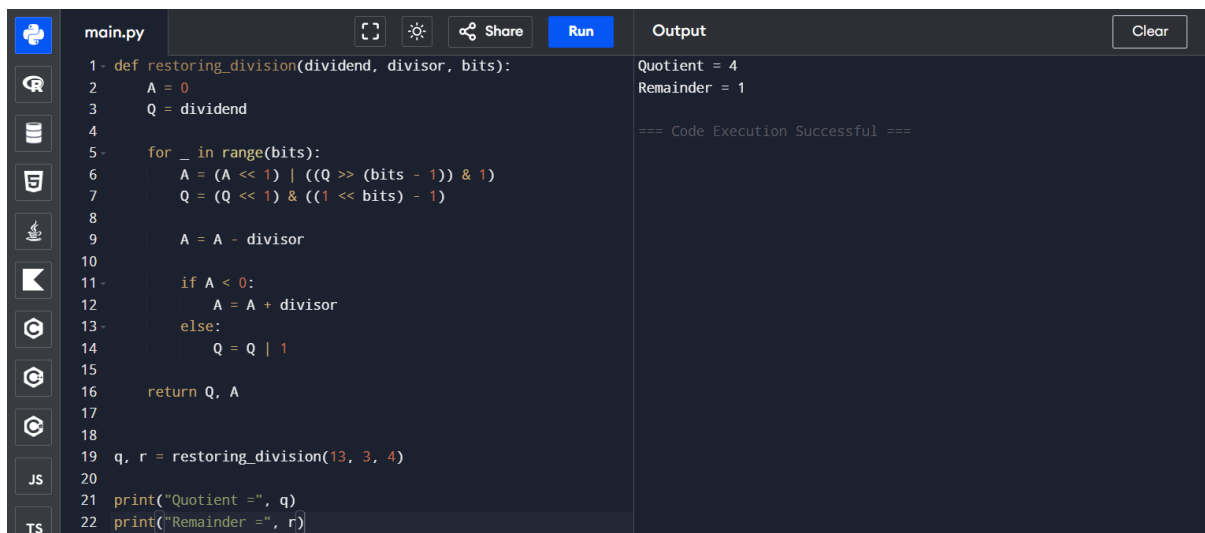


The screenshot shows a Python IDE with a file named 'main.py'. The code implements Booth's Multiplication algorithm. It takes a dividend (5) and a divisor (3) as input and returns the quotient (4) and remainder (1). The output window shows the execution results: '5 x 3 = 31' and '-5 x 3 = 49', followed by '=== Code Execution Successful ==='.

```
15 q0 = q & 1
16
17 if q0 == 1 and q_minus_1 == 0:
18     a = (a - m) & mask
19 elif q0 == 0 and q_minus_1 == 1:
20     a = (a + m) & mask
21
22 # Arithmetic shift of combined A,Q,Q-1
23 combined = (a << (n + 1)) | (q << 1) | q_minus_1
24 combined = combined >> 1
25
26 q_minus_1 = combined & 1
27 q = (combined >> 1) & mask
28 a = (combined >> (n + 1)) & mask
29
30 product = (to_signed(a) << n) | q
31 return to_signed(product >> (2 * n - 1)) * (1 << (2 * n
    - 1)) + (product & ((1 << (2 * n - 1)) - 1))
32
33
34 print("5 x 3 =", booth(5, 3, 4))
35 print("5 x 3 =", booth(5, 3, 4))
```

Output: 5 x 3 = 31
-5 x 3 = 49
=== Code Execution Successful ===

4.Restoring Division → Non-Restoring Division:

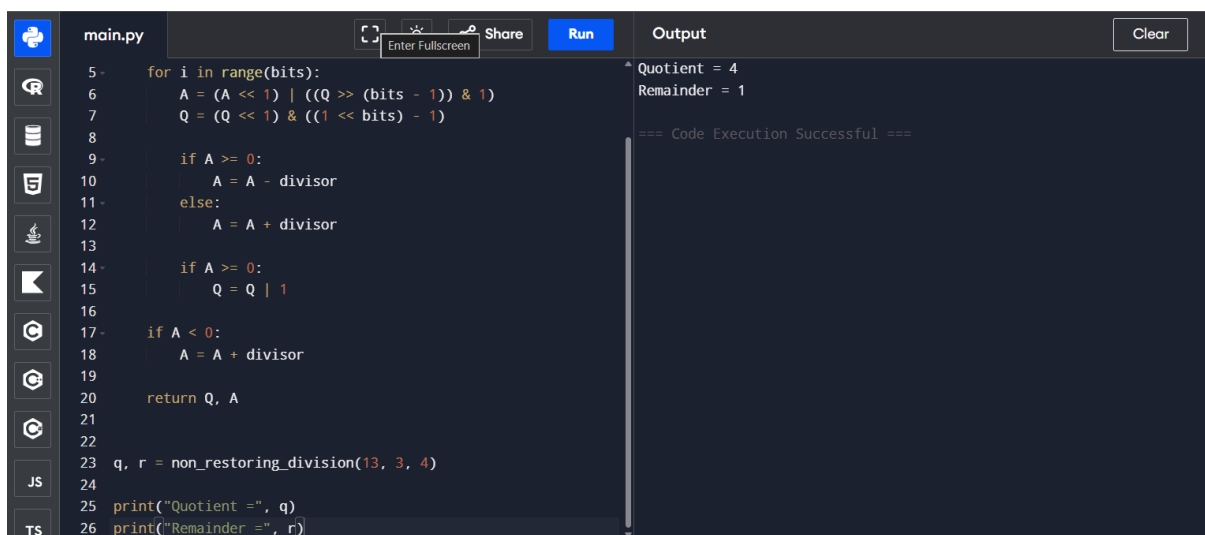


The screenshot shows a Python IDE with a file named 'main.py'. The code implements Restoring Division. It takes a dividend (13) and a divisor (3) as input and returns the quotient (4) and remainder (1). The output window shows the execution results: 'Quotient = 4' and 'Remainder = 1', followed by '=== Code Execution Successful ==='.

```
1 def restoring_division(dividend, divisor, bits):
2     A = 0
3     Q = dividend
4
5     for _ in range(bits):
6         A = (A << 1) | ((Q >> (bits - 1)) & 1)
7         Q = (Q << 1) & ((1 << bits) - 1)
8
9         A = A - divisor
10
11         if A < 0:
12             A = A + divisor
13         else:
14             Q = Q | 1
15
16     return Q, A
17
18 q, r = restoring_division(13, 3, 4)
19
20 print("Quotient =", q)
21 print("Remainder =", r)
```

Output: Quotient = 4
Remainder = 1
=== Code Execution Successful ===

Optimised Non-Restoring Version

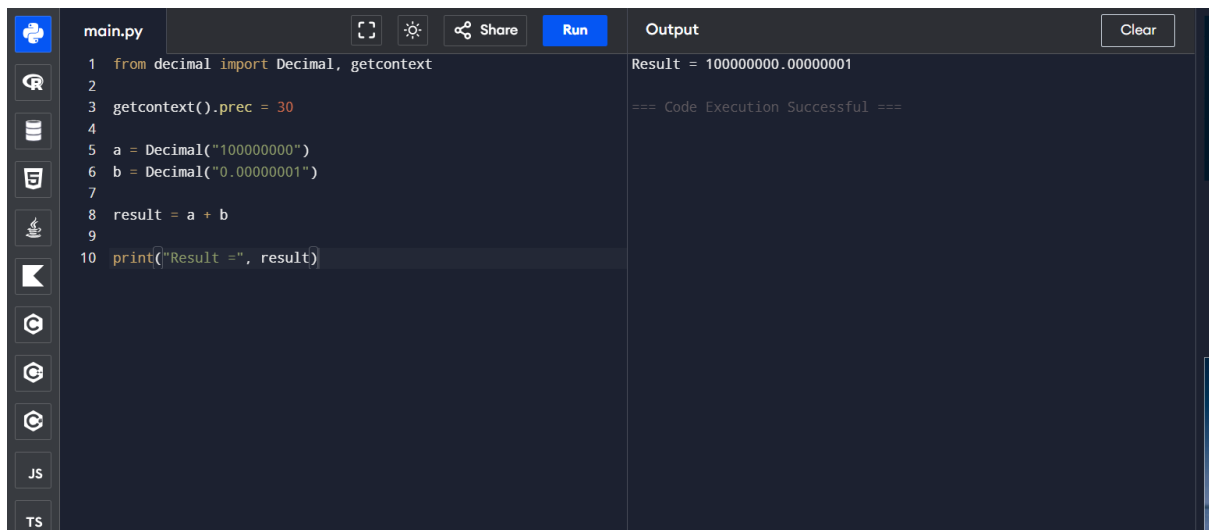


The screenshot shows a Python IDE with a file named 'main.py'. The code implements Optimised Non-Restoring Division. It takes a dividend (13) and a divisor (3) as input and returns the quotient (4) and remainder (1). The output window shows the execution results: 'Quotient = 4' and 'Remainder = 1', followed by '=== Code Execution Successful ==='.

```
5 for i in range(bits):
6     A = (A << 1) | ((Q >> (bits - 1)) & 1)
7     Q = (Q << 1) & ((1 << bits) - 1)
8
9     if A >= 0:
10         A = A - divisor
11     else:
12         A = A + divisor
13
14     if A >= 0:
15         Q = Q | 1
16
17     if A < 0:
18         A = A + divisor
19
20     return Q, A
21
22 q, r = non_restoring_division(13, 3, 4)
23
24 print("Quotient =", q)
25 print("Remainder =", r)
```

Output: Quotient = 4
Remainder = 1
=== Code Execution Successful ===

5. IEEE 754 Floating-Point Addition:



The image shows a code editor interface with a dark theme. On the left is a sidebar with icons for file explorer, search, and other tools. The main area is split into two panes. The left pane, titled 'main.py', contains a Python script. The right pane, titled 'Output', shows the result of running the script. The script uses the 'decimal' module to perform a floating-point addition with a precision of 30. The result is displayed as 'Result = 100000000.00000001'.

```
1 from decimal import Decimal, getcontext
2
3 getcontext().prec = 30
4
5 a = Decimal("100000000")
6 b = Decimal("0.00000001")
7
8 result = a + b
9
10 print("Result =", result)
```

Result = 100000000.00000001
=== Code Execution Successful ===